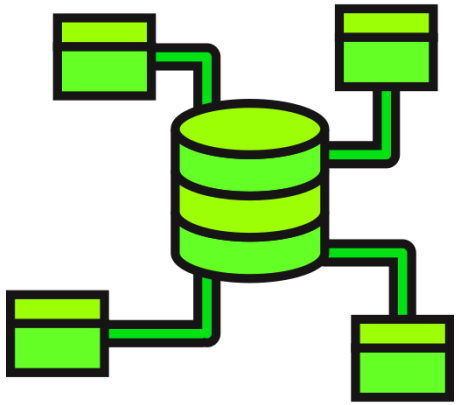
Реляционная модель

Реляционная модель

Создателем реляционной модели считается математик Эдгар Фрэнк Кодд.

Первопричиной возникновения нового по тем временам подхода к проектированию баз данных послужили существенные ограничения предыдущих моделей.

Модель данных не представляет существенного интереса для обычного пользователя, но без нее не обойтись разработчику БД. Ведь это не что иное, как чертеж будущей БД.

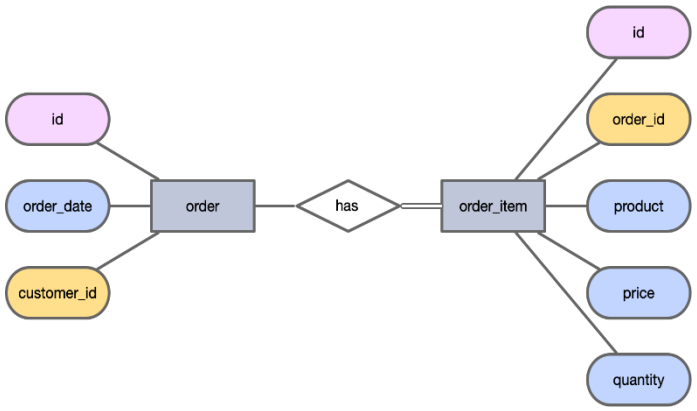


Сущность и атрибуты

На самом первом этапе проектирования будущей БД разработчик ломает голову над двумя ключевыми вопросами:

1. *Что подлежит хранению в БД?*
2. *Какие связи между данными следует поддерживать в БД?*

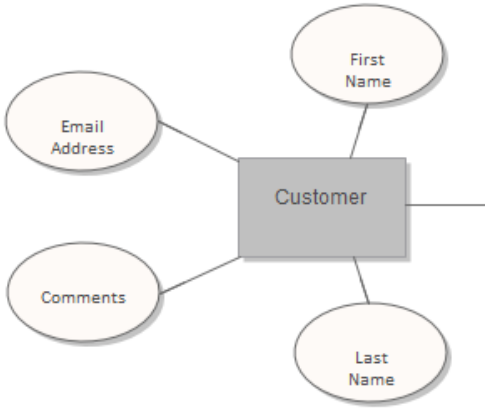
Для получения ответа на первый вопрос разработчик должен составить первоначальный перечень типов сущностей. Тип сущности (entity) – это класс однотипных объектов, о которых следует хранить информацию в БД.



Сущность и атрибуты

Экземпляр сущности – это то, о чем (или о ком) следует хранить информацию в БД. В реальной базе данных экземпляру сущности будет соответствовать одна строка таблицы.

Отличительные свойства сущности называют атрибутами.
Атрибуты могут быть простыми (*например, название города, в который летит самолет*) и составными (*т. е. включать в себя несколько простых атрибутов*).

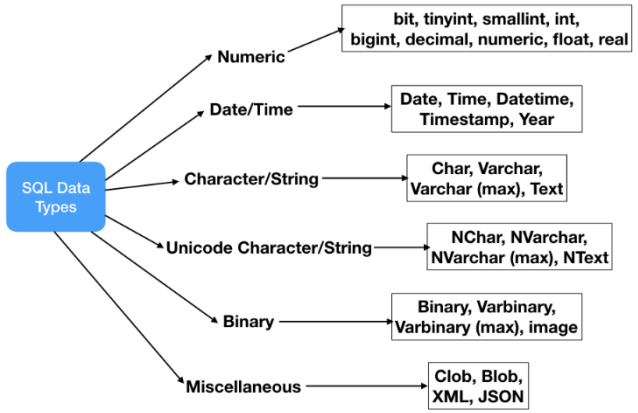


Тип данных и домен

Отправной точкой процесса построения любой системы хранения и обработки данных по праву считается выбор типа данных.

Домен в реляционной модели данных — тип данных, то есть множество допустимых значений

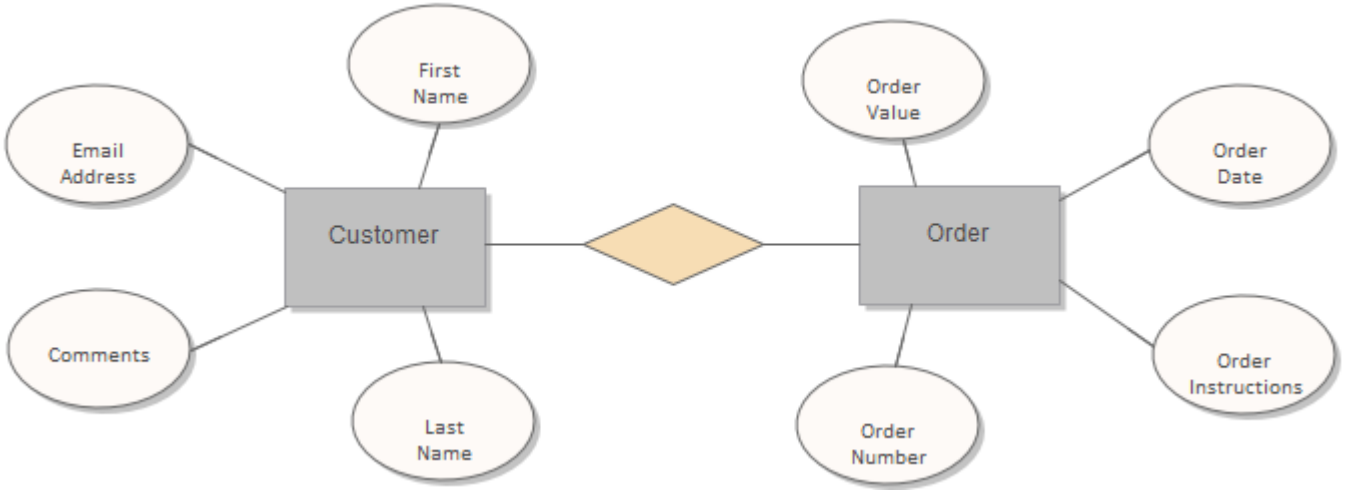
Тип данных определяет особенности физического хранения данных. Домен, опираясь на концепцию типа данных, несет дополнительную нагрузку – отвечает за поддержание логических правил описания данных. Благодаря доменным ограничениям в БД реализуется одно очень важное качество – доменная целостность данных.



Связь

Реляционная модель призвана не только описать перечень типов сущностей, но и отразить особенности взаимодействия между ними. Взаимодействие возникает там, где между типами сущностей проявляются отношения, выраженные в виде глагола.

Например покупатель оформил заказ. Здесь «покупатель» и «заказ» – это две сущности, а связь между ними просматривается в глаголе «оформил».



Связь

Различают три типа связи между сущностями:

- «один к одному»,
- «один ко многим»,
- «многие ко многим».

Один к одному

Появление в реляционной БД связи «один к одному» – крайне редкий случай. Чаще всего он свидетельствует о том, что разработчик не вполне корректно определился с атрибутами и вместо одного из атрибутов создал лишний тип сущности.

Один к одному значит, что строка в первой таблице связана с одной-единственной строкой во второй таблице.

Например, таблица работников с именами и должностью; а также таблица с данными для зарплаты. Каждому сотруднику соответствует одна строка с номером паспорта и банковским счётом, на который её начисляют. Id сотрудников одинаковый в обеих таблицах.

Работники

id	name	position
1	Анатолий	manager
4	Николай	director
10	Марина	developer

Данные для произведения оплаты

id	passport_data	bank_account
1	45540001	0000 0000 0000 0000
4	45450404	0000 0000 0000 0001
10	45444050	0000 0000 0000 0002

Этот тип связи встречается редко: его применяют, чтобы защитить данные. Например, к списку сотрудников могут получить данные все, а к данным на оплату — только бухгалтер.

Один ко многим

На практике наиболее распространена связь «один ко многим», например автор написал много книг, завод выпустил много автомобилей, в одном отделе работает много сотрудников.

Один ко многим — тип связи, когда каждая строка в одной таблице соответствует многим строкам в другой таблице.
Такой тип связи есть в примере с книгами. Автор может написать несколько книг, но у одной книги — только один автор.

Сотрудники

Ключ_сотрудника	Фамилия	Ключ_отдела
1	Данилова	2
2	Арбузов	2
3	Костенко	1
4	Костюков	3
5	Елецкова	1

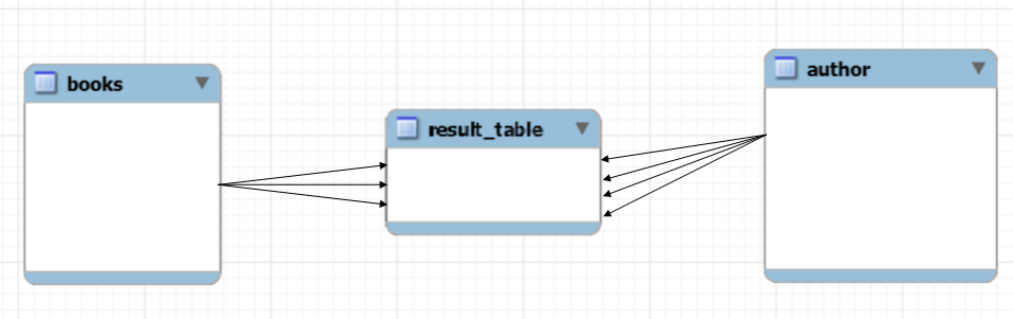
Отделы предприятия

Ключ_отдела	Название
1	Отдел кадров
2	Бухгалтерия
3	Отдел планирования

Многие ко многим

Связь «многие ко многим» встречается реже: офисы разных компаний размещены в разных городах, множество издательств опубликовало произведения различных авторов и т. п. Реляционная модель напрямую не поддерживает отношение «многие ко многим», поэтому его приходится разбивать на два отношения «один ко многим».

Многие ко многим — тип связи, когда несколько строк одной таблицы соответствуют нескольким строкам другой таблицы.



Одна книга могла быть написана несколькими авторами.
Автор мог написать несколько книг.

Многие ко многим

Например,

- Есть таблица authors с авторами и первичным ключом author_id;
- Есть таблица books с информацией о книгах и первичным ключом book_id

Как найти книги соответствующих авторов? Чтобы это было возможно, нужна ещё одна таблица authors_books: с её помощью сопоставляются таблицы authors и books.

authors

author_id	first_name	last_name
1	Александр	Пушкин
2	Лев	Толстой
3	Евгений	Петров
4	Иван	Тургенев
5	Илья	Ильф

authors_books

author_id	book_id
1	12
2	34
3	56
5	56
5	69

books

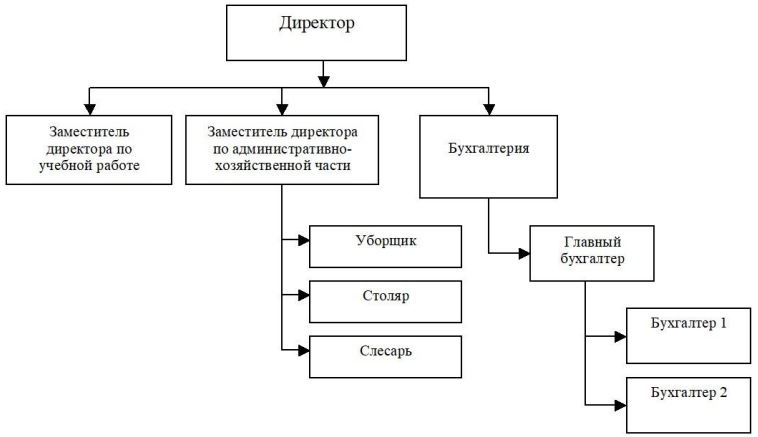
book_id	name
12	Капитанская дочка
34	Анна Каренина
56	Двенадцать стульев
66	Дети подземелья
69	Записные книжки

*Связь многие ко многим реализуется за счёт **связующей таблицы.** Она соединяет первичные ключи обеих таблиц.*

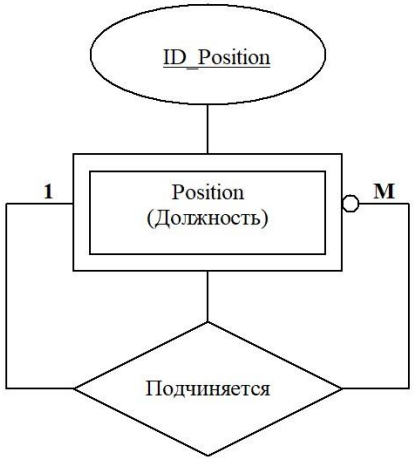
Рекурсивная связь

Реляционная модель позволяет связывать сущность саму с собой – задавать рекурсивную связь. Такой, на первый взгляд несколько необычный, способ взаимодействия оказывается весьма полезным при «выращивании» древообразных конструкций, например описывающих организационную структуру предприятия, завода или, скажем, университета.

Здесь четко просматривается иерархия, в которой одна и та же сущность может быть главной по отношению к подчиненным элементам и одновременно находиться в подчиненном состоянии к более высокому по рангу узлу.



Упрощенная организационно-штатная структура должностей учебного заведения



Рекурсивная связь на диаграмме ER-модели

ID_Position	ID_Parent	Position
1	NULL	Структура
2	1	Директор
3	2	Заместитель директора по учебной работе
4	2	Заместитель директора по административно-хозяйственной части
5	2	Бухгалтерия
6	4	Уборщик
7	4	Столяр
8	4	Слесарь
9	5	Главный бухгалтер
10	5	Бухгалтер 1
11	5	Бухгалтер 2

Таблица данных, которая соответствует ER-модели

Отношение

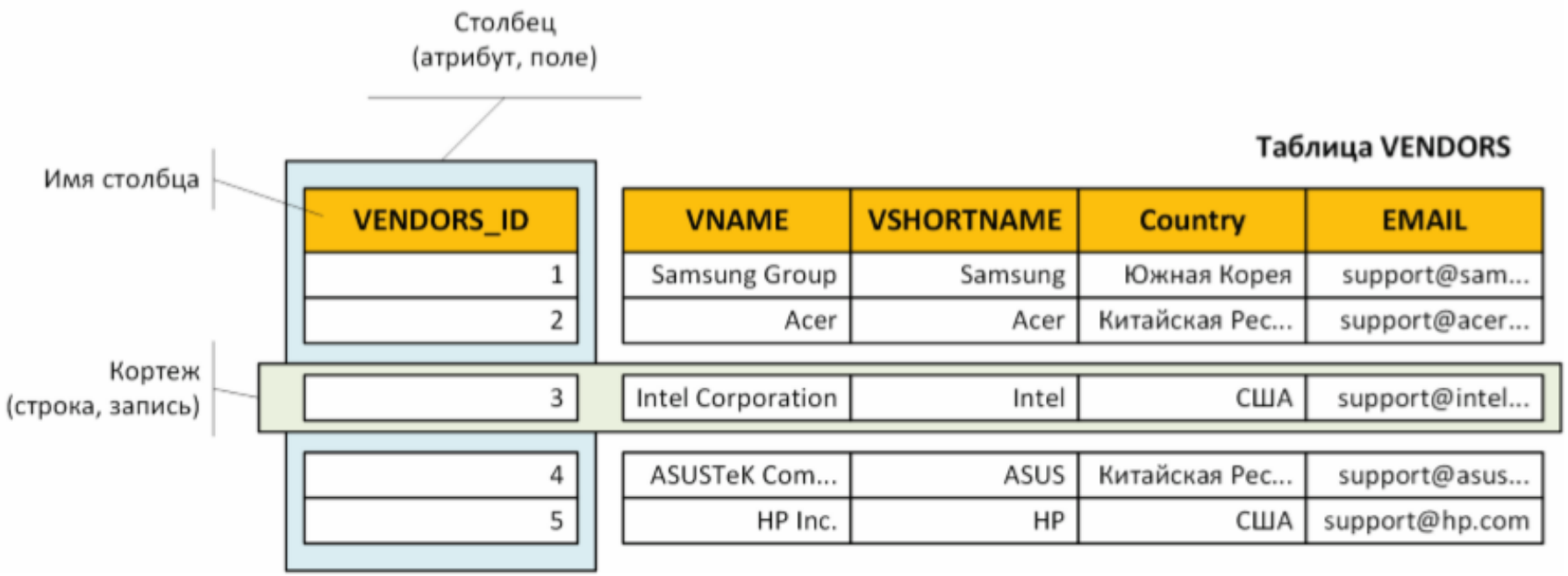
Терминология реляционных баз данных складывалась из нескольких областей знаний, поэтому в качестве синонимов к понятию «строка» можно услышать слова «запись» и «кортеж». У термина «столбец» также есть два тождественных понятия: «атрибут» и «поле». Ведя речь о таблице, вполне корректно применять термин «отношение».

Для того чтобы таблица (отношение) стала называться «реляционной», необходимо, чтобы она соответствовала ряду строгих требований:

- **уникальность имен столбцов внутри таблицы;**
- **каждая строка хранит информацию об отдельной сущности;**
- **строки таблицы отличаются друг от друга хотя бы единственным значением, что позволяет однозначно идентифицировать любую строку такой таблицы;**
- **на каждом пересечении строки и столбца хранится атомарное значение;**
- **строки и столбцы таблицы могут обрабатываться в любом порядке, независимо от хранящихся в них данных.**

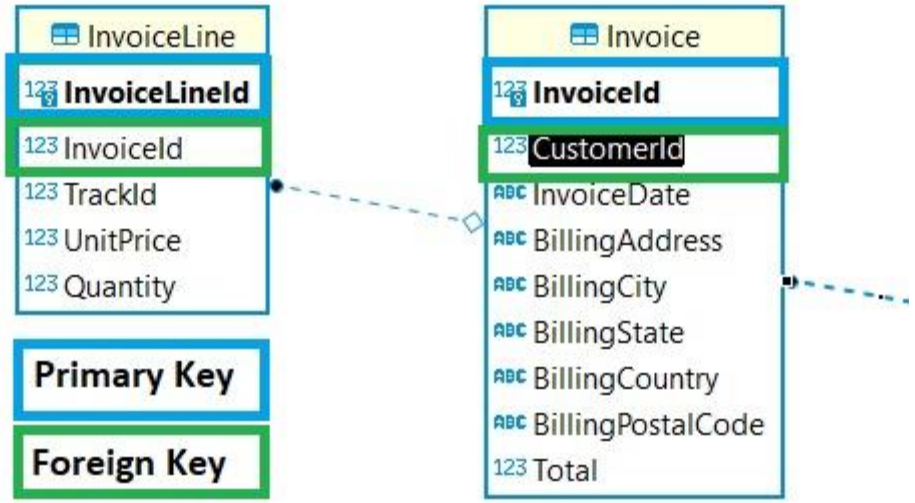
Отношение

Вся информация, собранная в реляционной БД, рассматривается пользователем как совокупность взаимосвязанных двумерных таблиц, в каждой из которых хранятся данные о строго определенном типе сущностей. Реляционная таблица состоит из строк и столбцов. Каждая строка содержит информацию о конкретной сущности.



Ключи

В составе любого реляционного отношения всегда должен присутствовать столбец (или группа столбцов), содержимое которого предназначается для хранения значений, способных уникальным образом идентифицировать строку таблицы.

Первичным ключом (PK, Primary Key) называют ключ, отвечающий за уникальную идентификацию строки в реляционной таблице.

Помимо первичных ключей, в таблицах активно применяется еще одна разновидность ключевого поля – внешний ключ (FK, Foreigner Key). Благодаря внешнему ключу мы получаем возможность строить реляционные связи между таблицами.